Workpath SDK AuthenticationAgent Overview

Product Group: HP Workpath SDK
Version: 1.6.3
Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

Table of Contents

1 Introduction 3
1.1 Document conventions 3

2 Authentication Agent 4
2.1 Authentication Process Initiation 4
2.1.1 UI Initiated Authentication 4
2.1.2 Non-UI Initiated Authentication 4
2.2 Authentication Agent Type 4
2.2.1 (Optional) Pre-prompt 4
2.2.2 Prompt 4

3 Authentication Process 5
3.1 Create Authentication Agent 6
3.2 Launch Authentication Agent 8
3.2.1 Sign-In Method on HP launcher 8
3.2.2 Sign-In Method by EWS 9
3.3 (Optional) Pre-prompt check 11
3.4 Prompt check 12
3.5 Switch to HP launcher 13

4 Authentication Information Generation 14

5 Authentication Agent Events 15

6 Authentication Agent Sample on a device 16
6.1 Initializing SDK 17
6.2 Generating Authentication Information 19
6.3 Requesting Sign-in with options 20

7 Authentication Agent with Preprompt Sample on a device 21
7.1 Extends AbstractAuthenticationService 22
7.2 Initializing SDK 23
7.3 Adding filter for getting events 24
7.4 Requesting Sign-in with options on onPrePrompt() 25

8 Legal notice 26

9 Support 27

1 Introduction

Authentication Agent provides use case scenario how to make authentication agent for authenticating front panel users on Workpath enabled devices.

1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- `Name` - Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

2 Authentication Agent

Workpath SDK defines API in `com.hp.workpath.api.access` package that can be called by Workpath enabled devices to authenticate with custom login information as walkup user.

2.1 Authentication Process Initiation

2.1.1 UI Initiated Authentication

When a walkup user presses the **Sign In** button and the user/device chooses the Authentication Agent as the Sign-In Method, the device will initiate the authentication process by invoking the Authentication Agent.

2.1.2 Non-UI Initiated Authentication

Non-UI initiated authentication is triggered by some out-of-band user-initiated event (entering a room, swiping a card, etc.) that is detected by the Authentication Agent.

NOTE: Since Workpath SDK v1.3, Non-UI Initiated Authentication has been supported. For getting more details, refer to AccessoryOverview

2.2 Authentication Agent Type

2.2.1 (Optional) Pre-prompt

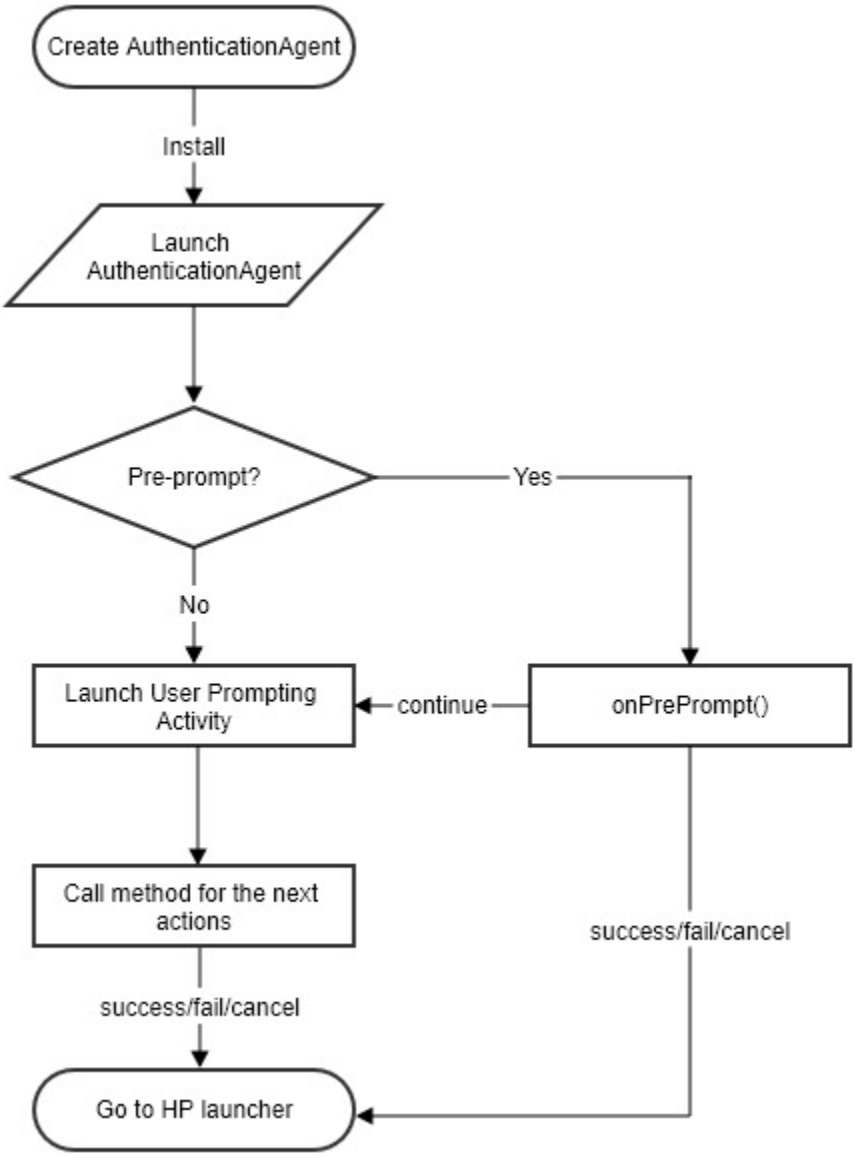
The Pre-prompt can be used as a way to check authentication before launching Prompt. It's configured as boolean value via HPKTool, and Authentication agent can decide the next actions in event with one of the options which is defined by `SignInAction.Action`.

2.2.2 Prompt

The Prompt is launched by device to load the activity specified by the Authentication Agent. Valid path of Prompt must be packaged with HPK file. If it's not, launching Prompt will be failed and UI will go back to HP launcher.

3 Authentication Process

When the device invokes a Proxy to authenticate a device front panel user, the Proxy will call the Auth Agent to perform the authentication

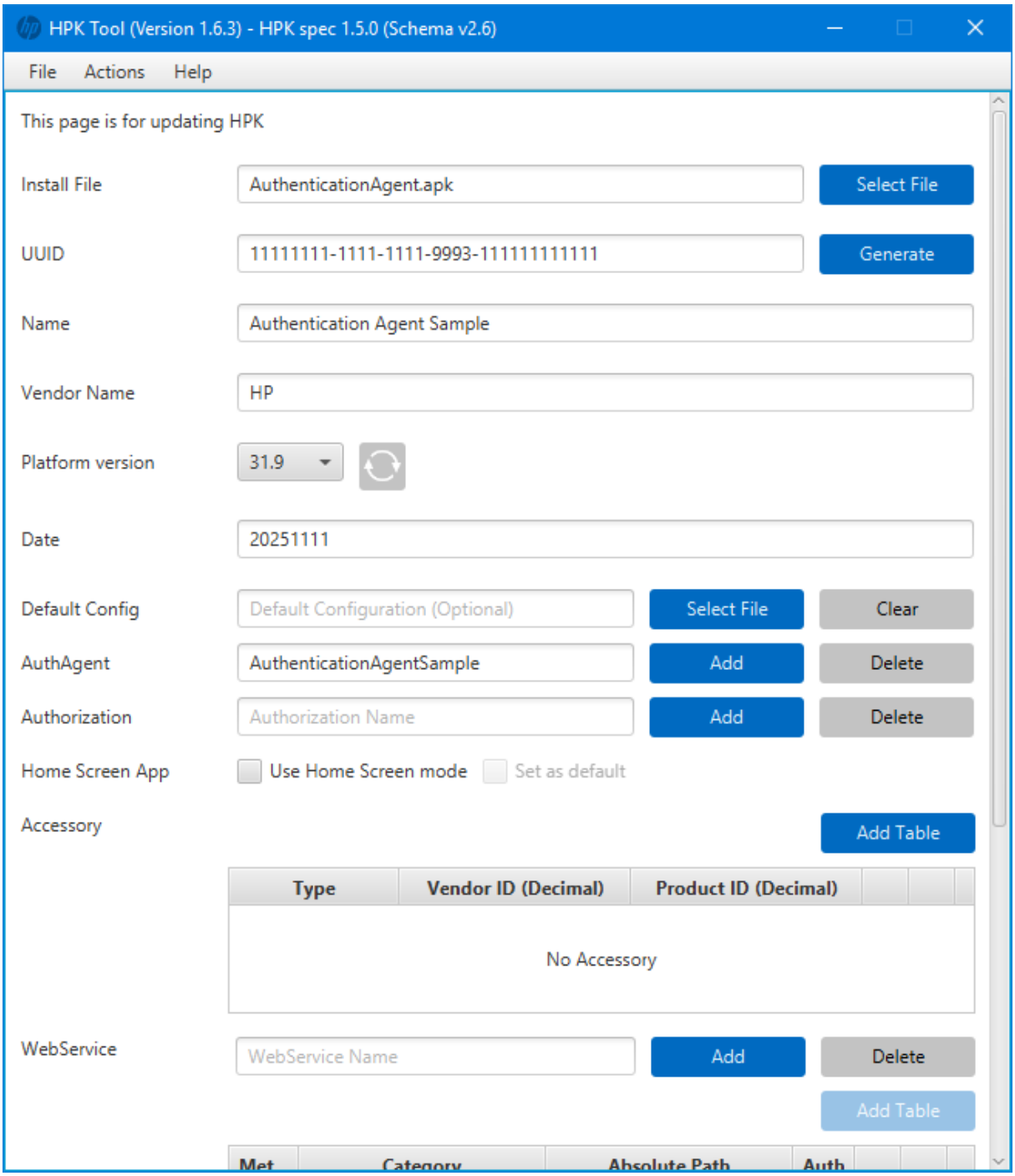


Authentication Agent Process

3.1 Create Authentication Agent

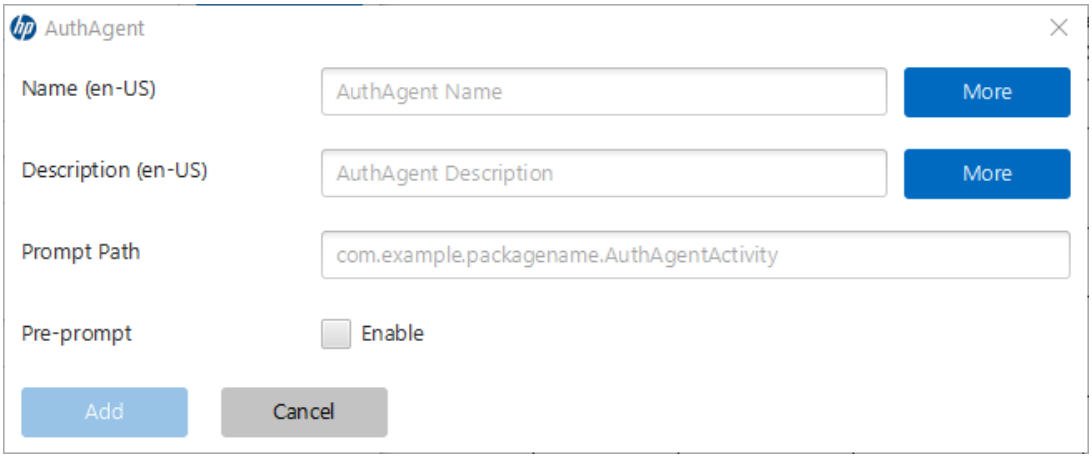
To enable Authentication Agent, HPK file has to include Authentication Agent information. HPKTool helps to create Authentication Agent as follows:

1. Open HPKTool.



The dialog to create HPK file

2. Click **Add**, then fill Auth Agent details.



An example for creating a AuthAgent

Fields		**Options**	**Description**
AuthAgent	Name	Required	Name of AuthAgent that will be shown on sign-in method lists. Default is English and multi-language is supported.
	Description	Required	Description of AuthAgent. Default is English and multi-language is supported.
	Prompt Path	Required	Prompt Path associated with AuthAgent. Value is defined by workpath application. Expected value is activity path.
	Pre-prompt	Optional	Flag to invoke pre-prompt service before launching Prompt.

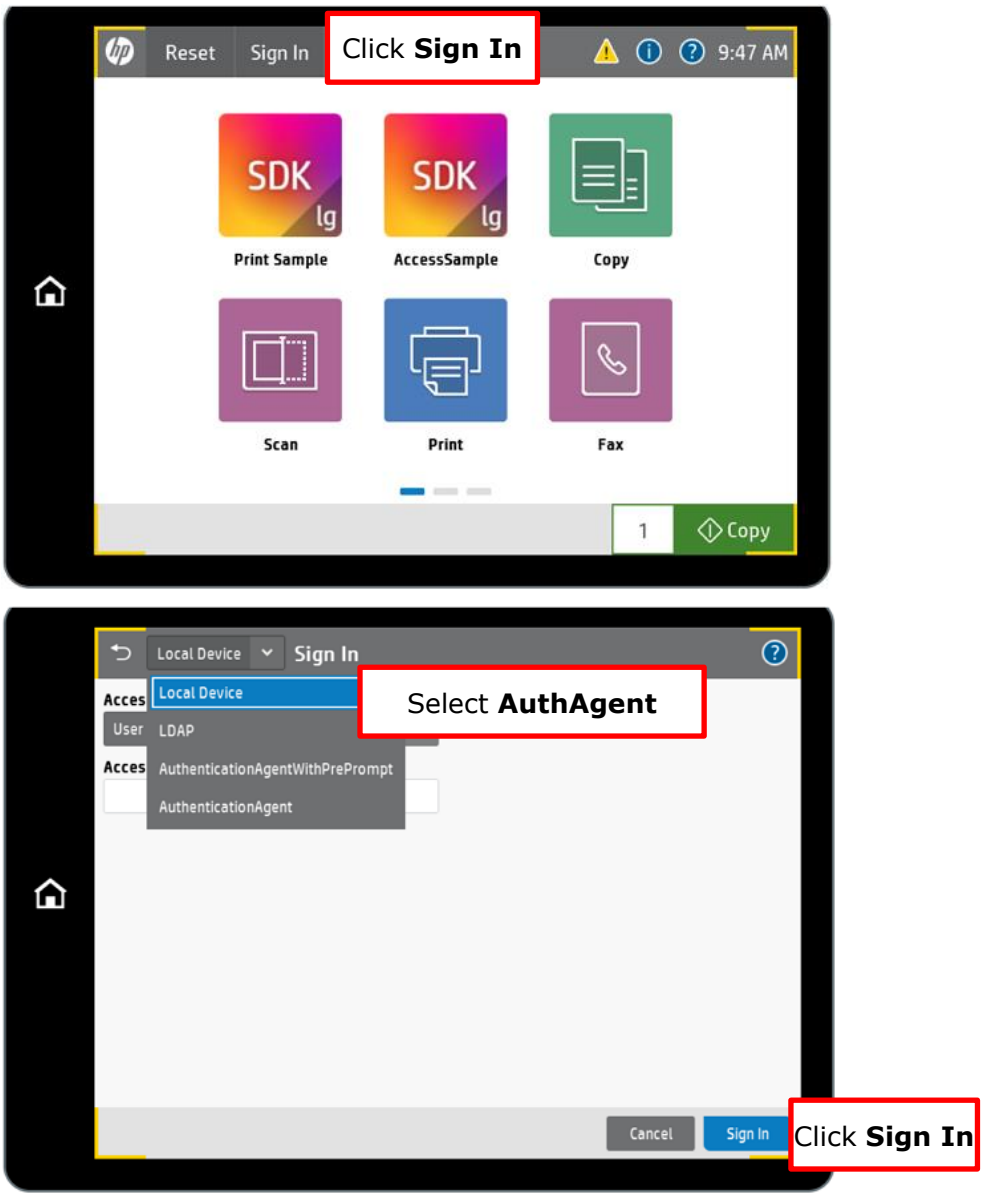
Details of AuthAgent

3. Install **HPK file** on a device.

3.2 Launch Authentication Agent

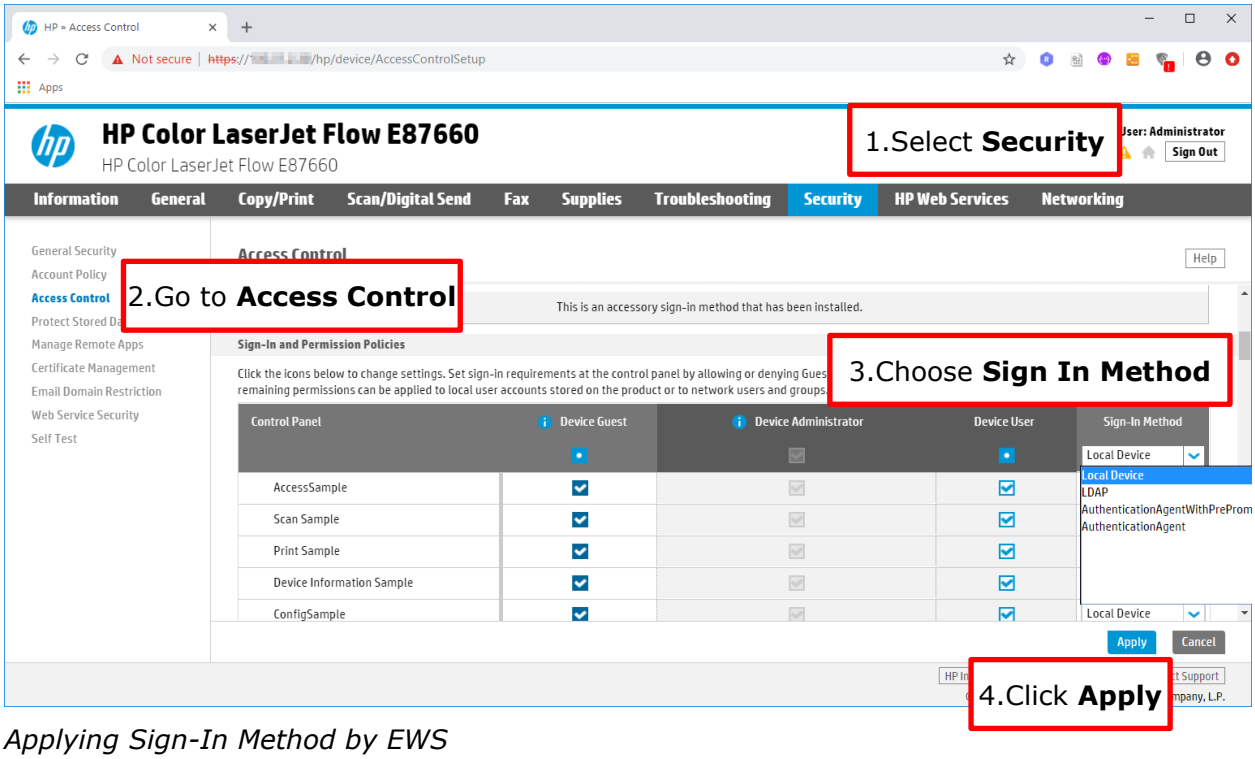
User/admin can launch Authentication Agent from Sign-In Method.

3.2.1 Sign-In Method on HP launcher



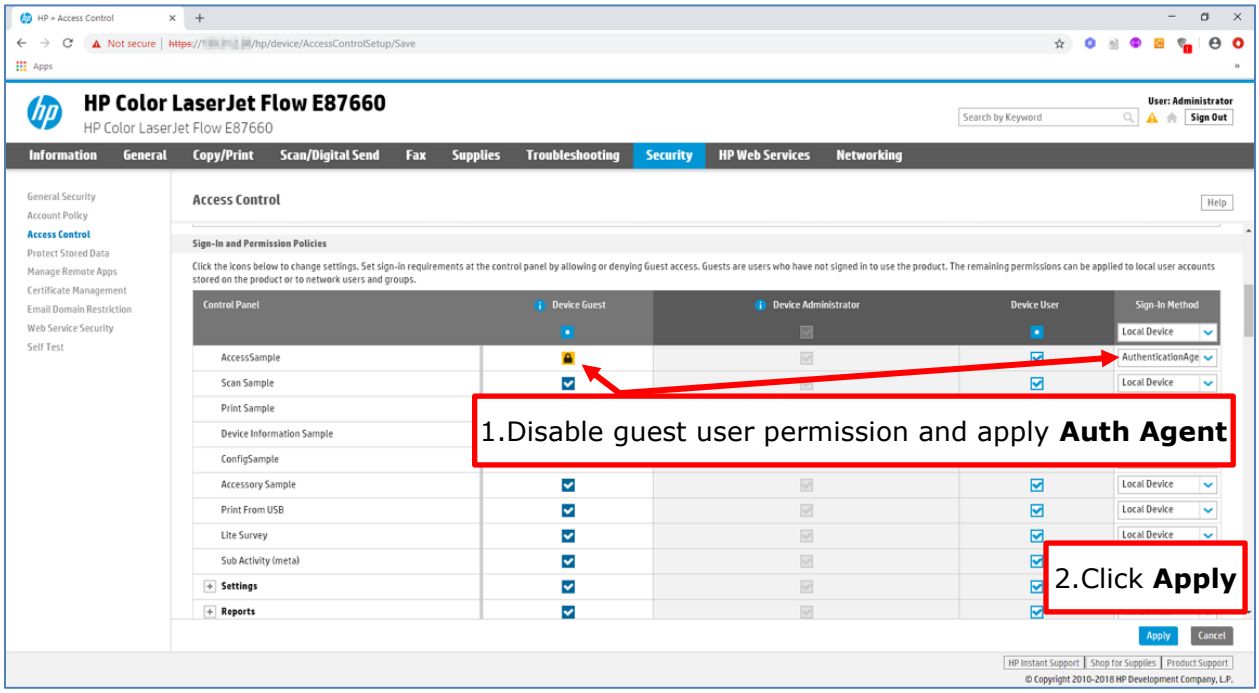
Sign-In Method on HP launcher

3.2.2 Sign-In Method by EWS

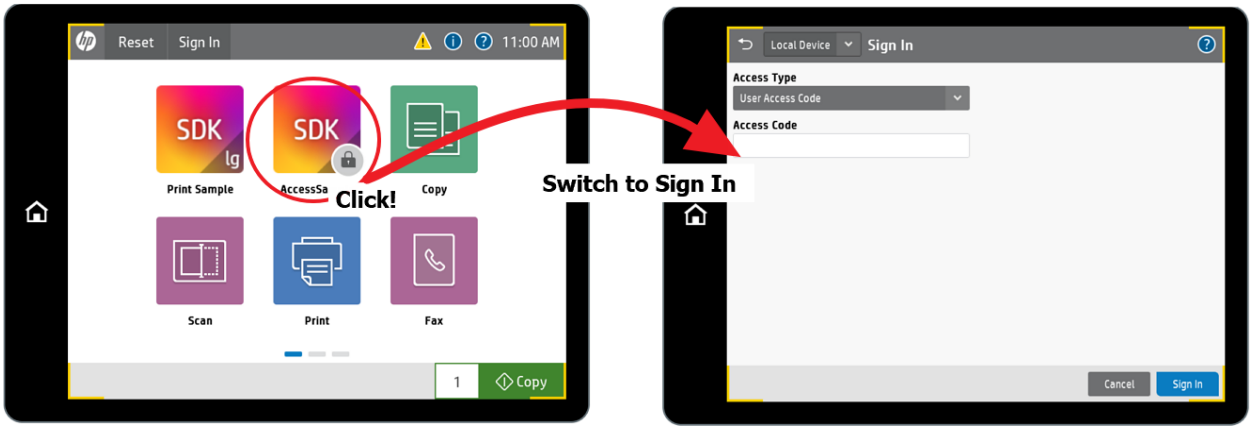


Applying Sign-In Method by EWS

This example shows how to apply Sign-In Method and work:



Applying Sign-In and Permissions Policies



Launching a Workpath application with Sign-In and Permissions Policies

3.3 (Optional) Pre-prompt check

When Authentication process is initiated, Workpath checks Pre-prompt option and sends the event to an agent. Authentication Agent which implements `AbstractAuthenticationService` can decide the next action with the one of the options which are defined by `SignInAction.Action` as follows:

SignInAction.Action	Caller	Next Page	Expected behavior
CONTINUE	Pre-prompt only	Prompt	The authentication process is incomplete and requires user prompting. Device will launch Prompt page.
SUCCESS	Pre-Prompt	HP launcher	The authentication process will be complete and succeeded. UI will switch to HP launcher with authenticated user information.
~~CANCEL~~ (Deprecated)	Pre-Prompt	HP launcher	The authentication process will be canceled. UI will go back to HP launcher.
HOME	Pre-Prompt	Home screen	The authentication process will be canceled. UI will go back to the home screen.
BACK	Pre-Prompt	Previous screen	The authentication process will be canceled. UI will go back to the previous screen.
FAIL	Pre-Prompt	HP launcher	The authentication process will be failed with message. UI will go back to HP launcher.

Sign-In action and expected behavior in Pre-prompt

Action.HOME and Action.BACK is supported since SDK 1.6.2

3.4 Prompt check

If Pre-prompt returns "CONTINUE" or Authentication Agent doesn't have Pre-prompt, Device will launch Prompt based on the path which is configured in HPK file. Authentication Agent can decide the authentication process with `SignInAction.Action` as follows:

SignInAction.Action	**Caller**	**Next Page**	**Expected behavior**
CONTINUE	Not available	Not available	Not available
SUCCESS	Prompt	HP launcher	The authentication process will be complete and succeeded. UI will switch to HP launcher with authenticated user information.
~~CANCEL~~ (Deprecated)	Prompt	HP launcher	The authentication process will be canceled. UI will go back to HP launcher.
HOME	Prompt	Home screen	The authentication process will be canceled. UI will go back to the home screen.
BACK	Prompt	Previous screen	The authentication process will be canceled. UI will go back to the previous screen.
FAIL	Prompt	HP launcher	The authentication process will be failed with message. UI will go back to HP launcher.

Sign-In action and expected behavior in Prompt

Action.HOME and Action.BACK is supported since SDK 1.6.2

3.5 Switch to HP launcher

When Authentication process is finished, UI will switch to HP launcher. Workpath SDK provides the `AccessService.getCurrentPrincipal()` method for getting authenticated user information.

4 Authentication Information Generation

The Authentication Agent can generate authentication information using one of the Builders which is defined by Workpath SDK as follows:

Builder	Details
AuthenticationAttributes.LdapBuilder	The Builder for creating AuthenticationAttributes of LDAP type to request sign in.
AuthenticationAttributes.NovellBuilder	The Builder for creating AuthenticationAttributes of Novell Directory Service type to request sign in.
AuthenticationAttributes.PinBuilder	The Builder for creating AuthenticationAttributes of Pin type to request sign in.
AuthenticationAttributes.WindowsBuilder	The Builder for creating AuthenticationAttributes of Windows type to request sign in.
AuthenticationAttributes.WindowsSmartCardBuilder	The Builder for creating AuthenticationAttributes of Windows Smart Card type to request sign in.
AuthenticationAttributes.OtherBuilder	The Builder for creating AuthenticationAttributes of Other type to request sign in.

AuthenticationAttributes builders by Workpath SDK

5 Authentication Agent Events

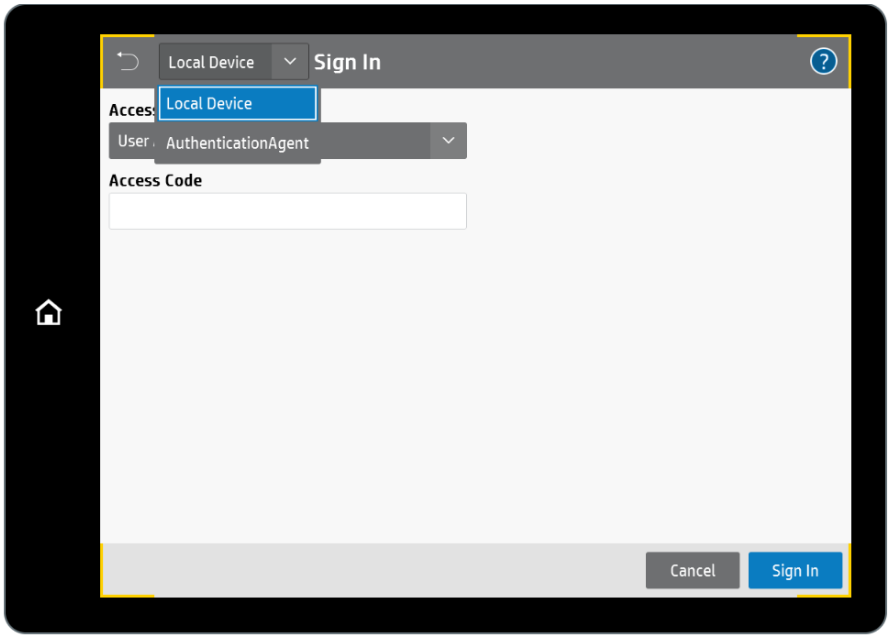
`AbstractAuthenticationService` by Workpath SDK defines methods to dispatch authentication events as follows:

Event	Details
onPrePrompt	Called to notify the client that prePrompt action is required to proceed with the authentication.
onSignIn	Called to notify the client that device has been authenticated successfully with user information.
onSignOut	Called to notify the client that SignOut operation is finished completely by a user request or authenticated user session is expired by timeout.

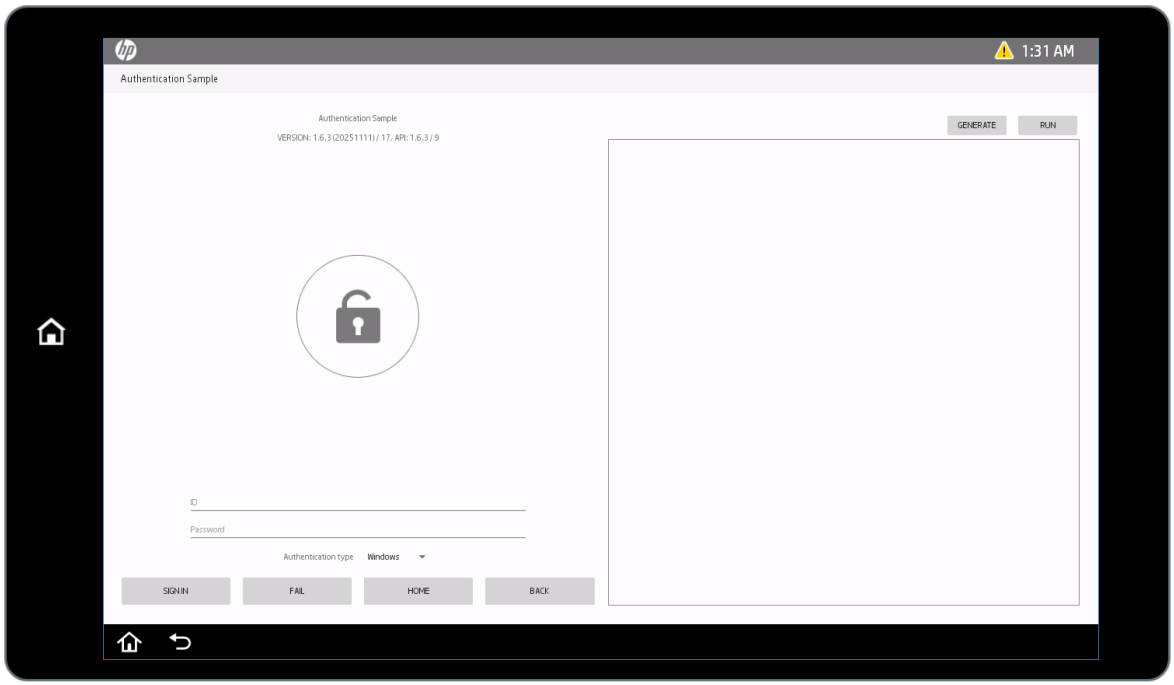
Authentication Agent Events by Workpath SDK

6 Authentication Agent Sample on a device

The Workpath SDK provides AuthenticationAgent sample and AuthenticationAgentWithPrePrompt sample to explain Authentication Agent feature. The AuthenticationAgent includes Sign-in activity:



An AuthenticationAgent in Sign-in methods



A Sign-in activity in AuthenticationAgent

6.1 Initializing SDK

1. Call `initialize()` method of Workpath SDK on resume event, and clear it on pause event. It is for calling initialization when an activity is switched. Refer to MainActivity.java.

```
1.  @Override
2.  protected void onResume() {
3.      super.onResume();
4.
5.      // call init task
6.      mInitializationTask = new InitializationTask(this);
7.      mInitializationTask.taskExecute();
8.  }
9.
10. @Override
11. protected void onPause() {
12.     super.onPause();
13.
14.     mInitializationTask.cancel();
15.     mInitializationTask = null;
16.
17.     ...
18. }
```

Initialization call in Activity Resume

2. Make a task and call `initialize()` method with catching exceptions. If Workpath SDK is not initialized, sdk exception or security exception will be thrown. Refer to InitializationTask.java.

```
1.  @Override
2.  public void run() {
3.      InitStatus status = InitStatus.NO_ERROR;
4.
5.      try {
6.          // initialize Workpath SDK
7.          Workpath.getInstance().initialize(mContextRef.get());
8.
9.          // Check if AccessService is supported
10.         if (!AccessService.isSupported(mContextRef.get())) {
11.             // AccessService is not supported on this device
12.             status = InitStatus.NOT_SUPPORTED;
13.         }
14.     } catch (SsdkUnsupportedException sue) {
15.         mThrowable = sue;
16.         status = InitStatus.INIT_EXCEPTION;
17.     } catch (SecurityException se) {
18.         mThrowable = se;
19.         status = InitStatus.INIT_EXCEPTION;
20.     } catch (Throwable t) {
21.         mThrowable = t;
22.         status = InitStatus.INIT_EXCEPTION;
23.     }
24.
25.
26.     onPostExecute(status);
27. }
```

Exception handling by initializationTask

6.2 Generating Authentication Information

Sample shows how to generate the authentication information using WindowsBuilder().

```
1.   private AuthenticationAttributes getWindowsData() throws Exception {
2.      String id = mIdEditText.getText().toString();
3.      String password = mPasswordEditText.getText().toString();
4.
5.      UserOverridesAttributes userOverridesAttributes = new UserOverridesAttributes.Build
    er()
6.              .addBccAddress(getString(R.string.bcc_address_email_01),getString(R.string.
    bcc_address_name_01))
7.              .addBccAddress(getString(R.string.bcc_address_email_02),getString(R.string.
    bcc_address_name_02))
8.              .addCcAddress(getString(R.string.cc_address_email_01),getString(R.string.cc
    _address_name_01))
9.              .addCcAddress(getString(R.string.cc_address_email_02),getString(R.string.cc
    _address_name_02))
10.             .setFrom(getString(R.string.from_address_email),getString(R.string.from_add
    ress_name))
11.             .addToAddress(getString(R.string.to_address_email_01),getString(R.string.to
    _address_name_01))
12.             .addToAddress(getString(R.string.to_address_email_02),getString(R.string.to
    _address_name_02))
13.             .setMessage(getString(R.string.email_message))
14.             .setSubject(getString(R.string.email_subject))
15.             .setFaxBillingCode(getString(R.string.fax_billing_code))
16.             .setFaxCompanyName(getString(R.string.fax_company_name))
17.             .build();
18.
19.     UserPreferencesAttributes userPreferencesAttributes = new UserPreferencesAttributes
    .Builder()
20.             .setAutoLaunchAppAccessPointId(getString(R.string.app_access_point_id))
21.             .setLanguageCode(getString(R.string.language_code))
22.             .build();
23.
24.     return new AuthenticationAttributes.WindowsBuilder()
25.             .setFullyQualifiedName(getString(R.string.value_fully_qualified_name, id))

26.             .setDisplayName(id)
27.             .setPassword(password)
28.             .setUserDomain(getString(R.string.value_domain))
29.             .setUserEmail(getString(R.string.value_user_email, id))
30.             .setUserName(id)
31.             .setUserPrincipalName(id)
32.             .setHomeFolderPath(getString(R.string.value_home_folder_path))
33.             .addUserProperty(getString(R.string.value_user_property_key_01), getString(
    R.string.value_user_property_value_01))
34.             .addUserProperty(getString(R.string.value_user_property_key_02), getString(
    R.string.value_user_property_value_02))
35.             .setUserOverridesAttributes(userOverridesAttributes)
36.             .setUserPreferencesAttributes(userPreferencesAttributes)
37.             .build();
38. }
```

Generating user information with WindowsBuilder()

6.3 Requesting Sign-in with options

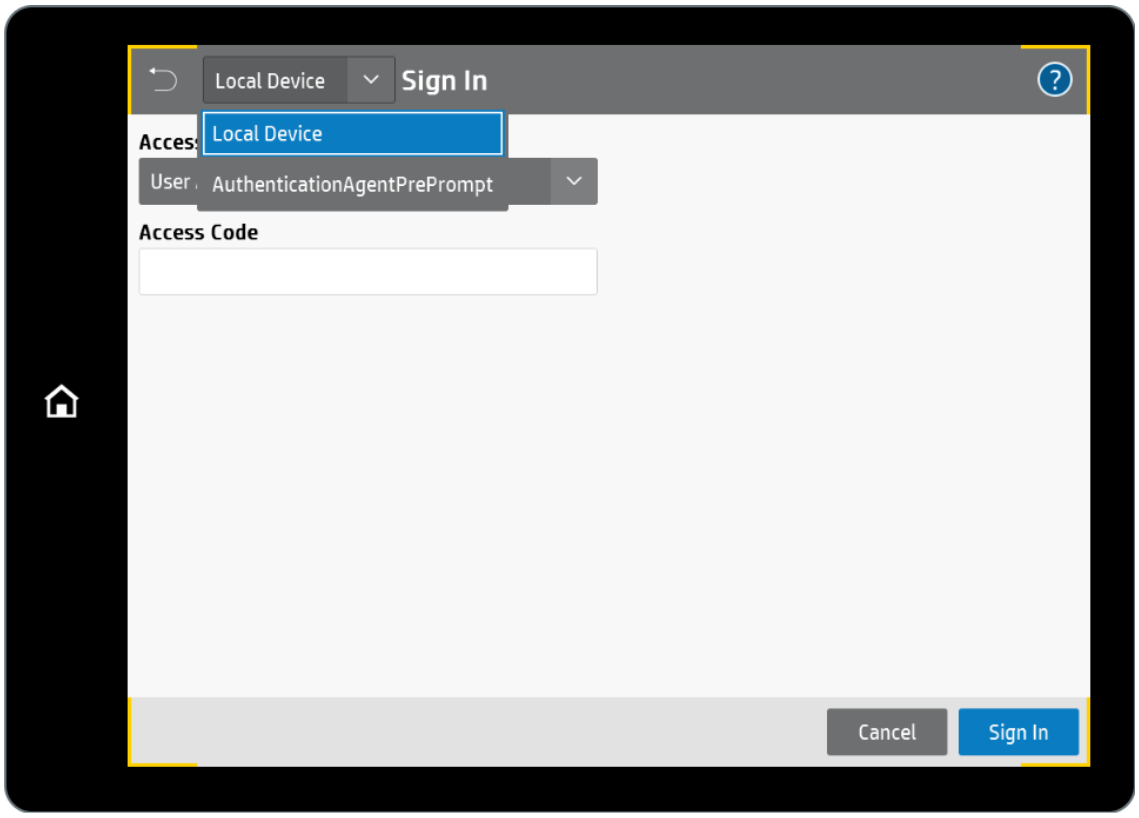
Sample calls signIn method with SignInAction to finish authentication process:

```
1.  View.OnClickListener buttonClickListener = new View.OnClickListener() {
2.      @Override
3.      public void onClick(View v) {
4.          if (TextUtils.isEmpty(mIdEditText.getText().toString())) {
5.              mIdEditText.setText("Tester");
6.          }
7.          try {
8.              Result result = new Result();
9.              if (v == mSignInButton) {
10.                 SignInAction signInAction = new SignInAction(SignInAction.Action.SUCCES
    S, null);
11.                 AccessService.signIn(AuthenticationActivity.this, signInAction, getWind
    owsData(), result);
12.             } else if (v == mFailButton) {
13.                 String failureMessage = "This is failure message from AuthenticationAge
    nt Sample.";
14.                 SignInAction signInAction = new SignInAction(SignInAction.Action.FAIL,
    failureMessage);
15.                 AccessService.signIn(AuthenticationActivity.this, signInAction, null, r
    esult);
16.             } else if (v == mHomeButton) {
17.                 SignInAction signInAction = new SignInAction(SignInAction.Action.HOME,
    null);
18.                 AccessService.signIn(AuthenticationActivity.this, signInAction, null, r
    esult);
19.             } else if (v == mBackButton) {
20.                 SignInAction signInAction = new SignInAction(SignInAction.Action.BACK,
    null);
21.                 AccessService.signIn(AuthenticationActivity.this, signInAction, null, r
    esult);
22.             }
23.
24.             if (result.getCode() != Result.RESULT_OK) {
25.                 showResult("AccessService.signIn()", result);
26.             }
27.
28.         } catch (Exception e) {
29.             showResult(e.getMessage());
30.         }
31.     }
32. };
```

Calling signIn method with SignInAction

7 Authentication Agent with Preprompt Sample on a device

The AuthenticationAgentWithPrePrompt includes pre-prompt sample codes.



An AuthenticationAgentWithPrePrompt in Sign-in methods

7.1 Extends AbstractAuthenticationService

Extend AbstractAuthenticationService to override Pre-prompt event. It'll be invoked by Authentication process. Refer to AuthenticationService.java.

```
 1. public class AuthenticationService extends AbstractAuthenticationService {
 2.
 3.     private static final String TAG = AuthenticationActivity.TAG + "/S";
 4.
 5.     /* Background task for Workpath SDK API initialization */
 6.     private InitializationTask mInitializationTask;
 7.     BlockingQueue<Boolean> initializedSDKQueue = new ArrayBlockingQueue<>(1);
 8.
 9.     @Override
10.     public void onCreate() {
11.         super.onCreate();
12.         initializedSDK();
13.     }
14.
15.     @Override
16.     public void onDestroy() {
17.         super.onDestroy();
18.
19.         if (mInitializationTask != null) {
20.             mInitializationTask.cancel();
21.             mInitializationTask = null;
22.         }
23.     }
24.
25.     @Override
26.     protected void onSignIn(Principal principal) {
27.         Log.i(TAG, "Received sign in event: " + principal.getUsername());
28.     }
29.
30.     @Override
31.     protected void onSignOut() {
32.         Log.i(TAG, "Received sign out event");
33.     }
34.
35.     @Override
36.     protected void onPrePrompt() {
37.         try {
38.             if (initializedSDKQueue.take()) {
39.                 Result result = new Result();
40.                 SignInAction signInAction = new SignInAction(SignInAction.Action.SUCCESS, null);
41.                 AccessService.signIn(AuthenticationService.this, signInAction, getWindowsData(),
result);
42.                 if (result != null && result.getCode() != Result.RESULT_OK) {
43.                     showLogResult("AccessService.signIn", result);
44.                     SignInAction failAction = new SignInAction(SignInAction.Action.FAIL,
result.getCause());
45.                     AccessService.signIn(AuthenticationService.this, failAction, null, null);
46.                 }
47.             }
48.         } catch (Throwable t) {
49.             showLogResult("AccessService.signIn " + t.getMessage());
50.         }
51.     }
52.
```

Extends AbstractAuthenticationService

7.2 Initializing SDK

If authentication process is initiated, `onPreprompt()` method will be called out-of-a activity. Therefore, authentication service should initialize SDK service in `onCreate()`. Refer to MainActivity.java.

```
 1. public class AuthenticationService extends AbstractAuthenticationService {
 2.
 3.     private static final String TAG = AuthenticationActivity.TAG + "/S";
 4.
 5.     /* Background task for Workpath SDK API initialization */
 6.     private InitializationTask mInitializationTask;
 7.     BlockingQueue<Boolean> initializedSDKQueue = new ArrayBlockingQueue<>(1);
 8.
 9.     @Override
10.     public void onCreate() {
11.         super.onCreate();
12.         initializedSDK();
13.     }
14.
```

Initialization call on onCreate()

7.3 Adding filter for getting events

Workpath SDK sends pre-prompt, sign in/sign out event through "com.hp.jetadvantage.link.api.action.AUTHENTICATION" filter. Add authentication filter in AndroidManifest.xml.

```
1.  <service android:name=".AuthenticationService"
2.      android:permission="com.hp.jetadvantage.link.permission.SERVICES_PERMISSION"
3.      android:exported="true">
4.      <intent-filter>
5.          <action android:name="com.hp.jetadvantage.link.api.action.AUTHENTICATION" />
6.      </intent-filter>
7.  </service>
```

Authentication filter

7.4 Requesting Sign-in with options on onPrePrompt()

Request Sign-in with user information on `onPrePrompt()`.

```
 1. @Override
 2. protected void onPrePrompt() {
 3.     try {
 4.         if (initializedSDKQueue.take()) {
 5.             Result result = new Result();
 6.             SignInAction signInAction = new SignInAction(SignInAction.Action.SUCCESS, null);
 7.             AccessService.signIn(AuthenticationService.this, signInAction, getWindowsData(),
result);
 8.             if (result != null && result.getCode() != Result.RESULT_OK) {
 9.                 showLogResult("AccessService.signIn", result);
10.                 SignInAction failAction = new SignInAction(SignInAction.Action.FAIL,
result.getCause());
11.                 AccessService.signIn(AuthenticationService.this, failAction, null, null);
12.             }
13.         }
14.     } catch (Throwable t) {
15.         showLogResult("AccessService.signIn " + t.getMessage());
16.     }
17. }
```

Calling signIn method with SignInAction on onPreprompt()

8 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

9 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <http://www.hp.com/go/solutions> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.